Journal of Systems Engineering and Electronics
Vol. PP, No. X, July 2025, pp.1 – 16

Game-based maneuver-fire decision model for UCAV air combat

DUAN Zekun[1], XU Genjiu[1,3,*], LI Zesheng[2], and MENG Binbin[4]

1. School of Mathematics and Statistics, Northwestern Polytechnical University, Xi'an 710072, China; 2. Unmanned System Research Institute, Northwestern Polytechnical University, Xi'an 710072, China; 3. Shaanxi Provincial Key Laboratory of Intelligent Game Theory and Information Processing at Higher Education Institutions, Xi'an 710072, China; 4. Intelligent Game and Decision Laboratory, National Innovation Institute of Defense Technology, Chinese Academy of Military Science, Beijing 100071, China

Abstract: In the Unmanned Combat Aerial Vehicle (UCAV) air combat operations, tactical positioning and precise attack timing emerge as critical determinants of mission success. The intricate interplay between these factors, coupled with the inherent dynamic nature of combat decision-making, poses significant challenges in developing effective air combat strategies. In this paper, we mainly focus on the one-on-one UCAV air combat scenario and proposes a dynamic maneuver-fire decision (DMFD) game model, which incorporates a utility function derived from a welfare function to comprehensively represent the decision-making processes involved in maneuvering and firing. To solve this game model, a predator-prey particle swarm optimization with refinement strategy space (PP-PSO-RSS) algorithm is proposed. The efficacy of our approach is validated through extensive numerical simulations, demonstrating superior performance in terms of win rate and real-time computational efficiency.

Keywords: game theory, air combat, intelligent algorithms.

DOI: 10.23919/JSEE.2025.000000

1. Introduction

Compared with unmanned aerial vehicles (UAVs), unmanned combat aerial vehicles (UCAVs) are capable of not only detection but also fire attack and intercept, as they are equipped with airborne weapons. They will play a central role in fighting for air supremacy in the future [1].

UCAV close air combat is a complex and dynamic process. UCAVs make maneuver decisions to maximize the situation advantage and minimize the situation threat from the other side. Meanwhile, each UCAV is equipped with limited airborne weapons (such as cannons) due to load limitations. In this case, UCAVs need to choose appropriate attack timing to avoid wasting weapons. In particular, the one-on-one scenario is the foundation of swarm air combat, making it essential to conduct research on air combat decision-making in such scenarios. In one-on-one scenarios, the UCAVs combat problem is generally decomposed into two sub-tasks: maneuver decision-making and fire decision-making.

The maneuver decision-making problem was first addressed by Austin et al. [2] using a matrix game approach in one-on-one air combat, with subsequent research [3–7] extending this to more complex scenarios. Methods such as dynamic programming [8,9], reinforcement learning [10–12], and influence diagrams [13,14] have also been applied. In contrast, fire decision-making involves weapon configuration [15] and fire strategy selection [16,17], with a focus on resource allocation and target selection. Importantly, in real combat, the timing of weapon use is a critical factor that can decisively influence the outcome. This issue is more complex than static resource allocation, as it is a dynamic multi-step process influenced by maneuver strategies. Additionally, the interdependence between maneuver and fire strategies further complicates the problem.

In adversarial environments, the coordination of maneuver and fire strategies impacts overall combat effectiveness. Reinforcement learning [18,19] and rule-based approaches [20] have shown promise in joint maneuver-fire decision-making. Compare with these approaches, game theory-based approach fundamentally emphasizes the interdependence of strategies among the participants. In complex battlefield environments where UCAVs may lack complete information about the oppo-

Manuscript received March 03, 2023.
*Corresponding author.
This work was supported by the National Natural Science Foundation of China (72071159), the National Key R&D Program of China (2021YFA1000402), and the Open Research Subject of State Key Laboratory of Intelligent Game (ZBKF-24-11).

nent, game theory's equilibrium strategies allow UCAVs to anticipate and respond to optimal moves, ensuring each UCAV maximizes its utility regardless of the opponent's actions, thus providing a robust decision-making framework. Therefore, this paper explores dynamic joint maneuver-fire decision-making problem within the game theory framework.

Linear programming is a classic method to calculate the Nash equilibrium, however, it is proved that this method is PPAD-complete [20,21], which is limited by the time-consuming problem when facing large strategy space. Heuristic algorithms are robust and efficient methods to approximate exact Nash equilibriums. Particle swarm optimization (PSO) algorithm is a heuristic global optimization algorithm based on bird flocking, proposed originally by Kennedy and Eberhart [22] in 1995. In this algorithm, the particle updates its speed according to the individual optimal position and global optimal position. However, this algorithm might trap local optima when it comes to complex, high-dimensional problems that have a lot of local optimal solutions. In order to solve this problem, the predator-prey particle swarm optimization (PP-PSO) algorithm was proposed in 2006 [23], dividing the particle into predator and prey inspired by the pursuit-escape phenomenon between schools of sardines and killer whales.

However, different particles have different speed update methods in the PP-PSO algorithm, and the speed update methods of particles depend on not only the individual optimal position and the global optimal position, but also the group optimal position. This changes result in an increase in convergence time when solving the complex high-dimension problem compared with the PSO algorithm. Facing with the rapidly changing battlefield environment, real-time strategy is critical to gain a competitive advantage. Hence, the PP-PSO algorithm requires optimization to enhance its real-time computational efficiency.

Driven by the aforementioned problems, this paper addresses the maneuver and fire decision-making problem in one-on-one air combat by proposing a game-based joint decision model for maneuver-fire coordination. Additionally, we present a modified PP-PSO algorithm to compute the MSNE of the proposed model. In conclusion, the main contributions of this paper are as follows:

(1) The dynamic maneuver-fire decision (DMFD) game model proposed in this paper simultaneously involves the maneuver and fire dynamic decision-making processes and their coupling relationships in air combat scenarios. Compared to the general matrix game model, the DMFD game model is more comprehensive and further enhances the completeness of the decision-making chain, which is more suitable for general air combat scenarios.

(2) The PP-PSO-RSS algorithm presented in this paper modifies the PP-PSO algorithm, reducing the average computational time nearly 83%. This modification further improves the computation speed of the PSO-based algorithm, enabling this algorithm to meet the real-time requirements of air combat.

(3) By conducting extensive numerical simulations, we demonstrate the superior performance of our method and analyze its effectiveness. Compared with game-based algorithms and reinforcement learning-based algorithms, our method has a higher winning rate and dominant rate under different initial conditions, which shows the efficiency of our method.

The reminder of the paper is organized as follows: Section 2 introduces the dynamical model of UCAVs. Section 3 describes the elements of the comprehensive advantage function. Section 4 proposes the objective function. Section 5 constructs the DMFD game model. Section 6 proposes the PP-PSO-RSS algorithm. Section 7 performs several experiments to illustrate the effectiveness and superiority of the proposed method. Section 8 concludes the paper.

2. UCAV dynamical model

2.1 The dynamical model of UCAVs

The position of single UCAV in 3-dimensional space is displayed in Fig. 1.

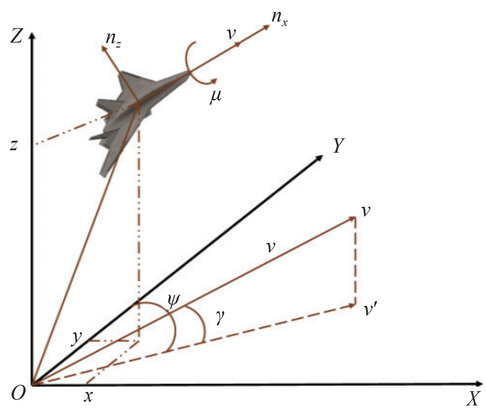


Fig. 1 Diagram of the UCAV's motion in three-dimensional space

x, y, and z define the position of aircraft. $\dot{x}$, $\dot{y}$, and $\dot{z}$ represent the derivative of position with respect to time, that is, the speeds in the three directions. g is the gravity. γ is the track angle, representing the angle between the velocity and the horizontal plane. ψ is the heading angle, representing the angle between the projection of velocity on the horizontal plane and the y-axis.n_x and n_z are the longitudinal load factor and normal load factor, respectively. μ is the roll angle around the velocity.

In this paper, we use the 3-degree-of-freedom dynamic model in Eq. (1) [24–26], the three-dimensional spatial kinematic equations of UCAVs in the ground inertial coordinate system, as the motion frame of aircrafts. It is worth noting that our research focuses on game-theoretic decision-making, and our approach can be extended to more complex dynamic systems to accommodate real-world air combat scenarios.

$$\begin{cases} \dot{x} = v\cos\gamma\sin\psi \\ \dot{y} = v\cos\gamma\cos\psi \\ \dot{z} = v\sin\gamma \\ \dot{v} = g(n_x - \sin\gamma) \\ \dot{\gamma} = g/v(n_z\cos\mu - \cos\gamma) \\ \dot{\psi} = \dfrac{gn_z\sin\mu}{v\cos\gamma} \end{cases} . \quad (1)$$

In Eq. (1), the first three equations form the motion equations of UCAVs and the last three equations form the dynamic equations of UCAVs. The control variables of the above system consist of (n_x,n_z,μ), representing the maneuver strategy of the UCAV, and the outputs of above system are composed of $(\dot{x},\dot{y},\dot{z})$[24].

In this paper, there are mainly three model assumptions to simplify our model. Firstly, we used a 3 -degree-of-freedom dynamic model, which reduces the complexity of the dynamic model but can capture the essential motion characteristics of UCAVs in air combat. Additionally, we assume that the position and orientation angles of both UAVs are fully known to each other. In reality, this information can be obtained through advanced sensing technologies. However, this paper primarily focuses on the utilization of this information, rather than how it is obtained. Finally, we adopt a discrete action space, where it is assumed that each UCAV selects one of the elemental maneuvers shown in Table 1. This assumption is widely used in air combat scenarios.

Table 1 Actions for maneuver control and fire attack control

Action	No.	Maneuver	n_x	n_z	μ
Maneuver Action	am_1	Forward maintain	0	1	0
	am_2	Forward accelerate	2	1	0
	am_3	Forward decelerate	−1	0	0
	am_4	Left turn maintain	0	8	−arccos(1/8)
	am_5	Left turn accelerate	2	8	−arccos(1/8)
	am_6	Left turn decelerate	−1	8	−arccos(1/8)
	am_7	Right turn maintain	0	8	arccos(1/8)
	am_8	Right turn accelerate	2	8	arccos(1/8)
	am_9	Right turn decelerate	−1	8	arccos(1/8)
	am_{10}	Upward maintain	0	8	0
	am_{11}	Upward accelerate	2	8	0
	am_{12}	Upward decelerate	−1	8	0
	am_{13}	Downward maintain	0	8	π
	am_{14}	Downward accelerate	2	8	π
	am_{15}	Downward decelerate	−1	8	π
Action	**No.**	**Fire attack choice**	**Control value**		
Fire Action	af_1	Attack	1		
	af_2	Keep	0		

2.2 Relative position of UCAVs in one-on-one air combat

In this paper, the UCAV from the red side is denoted by r, and the UCAV from the blue side is denoted by b.

The position relationship of the one-on-one scenario is shown in Fig. 2. In this figure, d_{rb} is the relative distance between the red and the blue. The line of sight refers to the direction pointing from itself to the opponent. $\boldsymbol{v}_r$ and $\boldsymbol{v}_b$ are the velocity vectors of r and b, respectively. θ_r is the included angle between the direction of the red's velocity and line of sight, named sight angle of r.θ_b is the included angle between the direction of the blue's velocity and line of sight, named sight angle of b.

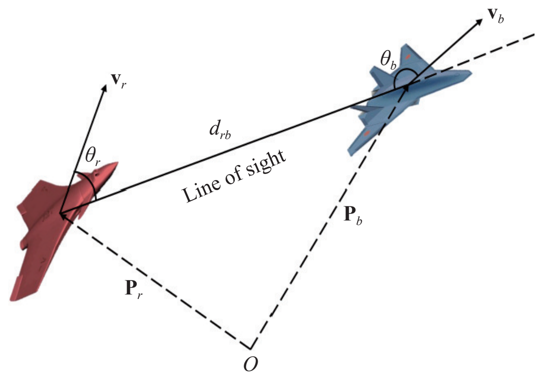


Fig. 2 Schematic of relative angle and distance

The relative distance is denoted by

$$d_{rb} = \boldsymbol{P}_r - \boldsymbol{P}_b = \sqrt{(x_r - x_b)^2 + (y_r - y_b)^2 + (z_r - z_b)^2}. \tag{2}$$

$\theta_r \in [0, \pi]$ and $\theta_b \in [0, \pi]$ are given by

$$\begin{cases} \theta_r = \arccos \dfrac{\boldsymbol{v}_r \cdot (\boldsymbol{P}_b - \boldsymbol{P}_r)}{v_r \|\boldsymbol{P}_b - \boldsymbol{P}_r\|}, \\ \theta_b = \arccos \dfrac{\boldsymbol{v}_b \cdot (\boldsymbol{P}_r - \boldsymbol{P}_b)}{v_b \|\boldsymbol{P}_r - \boldsymbol{P}_b\|}, \end{cases} \tag{3}$$

where the symbol $\cdot$ represents the inner product of two vectors, $\|\cdot\|$ is the modulus of a vector.

2.3 Fire area and capture area from the perspective of the red side

The fire area defines a reasonable area for one side to use weapons and evaluate the timing to attack the opponent.

Referring to Fig. 3, a schematic representation of the fire area for r is shown in a two-dimensional plane. When the velocity direction of b belongs to the light gray region, its head points to r. It is considered that r is threatened and the weapon is used for defense. Conversely, when the velocity direction of b belongs to the dark gray region, its tail is towards r, and it is considered that r will use the weapon for attack. Obviously, regardless of which side the velocity direction of b is located on, as long as the velocity direction of r falls within the semicircular region depicted in Fig. 3 and the distance between the two sides is less than the maximum attack distance, effective attack and defense can be achieved.

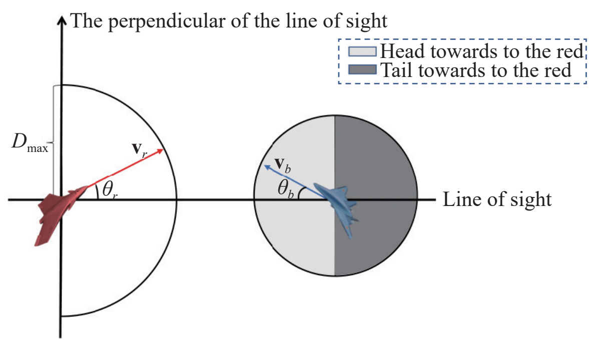


Fig. 3 The analysis of fire attack condition

Hence, the fire condition of r is defined as $c_{\text{fire}} = \{d_{rb} \leqslant D_{\max} \text{ and } \theta_r \leqslant \pi/2\}$. $D_{\max}$ is the maximum attack distance. This condition describes a hemisphere area Ω_{fire}, called the fire area, which is displayed in Fig. 4.

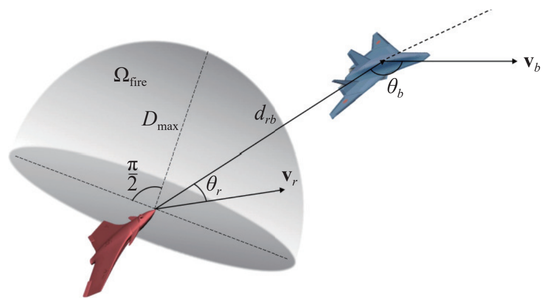


Fig. 4 The fire area of the UCAV

The capture area refers to the damage area caused by one side to the opponent and affects the degree of damage to the opponent. c_{capt} is the capture condition of r, which is given by $c_{\text{capt}} = \{d_{rb} \leqslant D_{\text{best}} \text{ and } 0 \leqslant \theta_r \leqslant \varphi_m\}$, resulting in a conical area Ω_{capture}. D_{best} is the optimal attack distance and φ_m is the maximum off-axis angle. We call this conical area as capture area of the weapon, which is shown in Fig. 5.

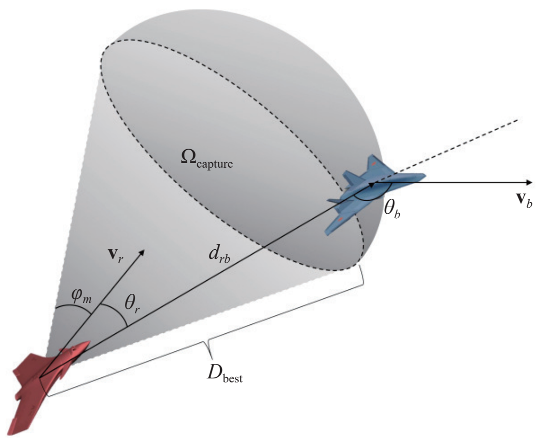


Fig. 5 The capture area of the weapon

In the same way, we can define the fire condition and capture condition of b, and describe the corresponding fire area and capture area.

3. The evaluation of situation

The purpose of the maneuver is to force the target into the aircraft's attack range while avoiding the aircraft from entering the target's attack range, so that it can gain an advantageous position in any situation [27]. In other words, the situation is determined by the maneuver decisions of aircraft. Generally, the situation is assessed by a function incorporating diverse factors [28–30]. In this paper, we select the angle, speed, distance, and altitude to characterize the situation. In the following contents, we define and analyze the corresponding advantage functions from the perspective of r, the related functions of b can be obtained in a similar way.

3.1 Angle advantage function

In 3-dimensional space, we mainly use the angle between the line of sight and the velocity to describe the angle advantage of the UCAV. In this paper, the angle advantage function of r is denoted by

$$F_r^a = 1 - \frac{\theta_r + (\pi - \theta_b)}{2\pi}. \tag{4}$$

Similarly, we can get the angle advantage of b.

$$F_b^a = 1 - \frac{(\pi - \theta_r) + \theta_b}{2\pi}. \tag{5}$$

Obviously, we have $F_r^a + F_b^a = 1$.

3.2 Speed advantage function

A higher speed enhances the UCAV's initiative, making it easier to approach or escape from the opponent. In this paper, we use the speed advantage function defined in [3], and the speed advantage function of r is denoted by

$$F_r^v = \mathrm{e}^{-|v_r - v_{r^*}|/v_{r^*}}, \tag{6}$$

where $v_r = \|\boldsymbol{v}_r\|$ is the scalar speed of the red and $|v_{r^*}|$ is the ideal speed of the red defined as follows:

$$v_{r^*} = v_b + (v_{\max} - v_b)\left(1 - e^{-|D_{\max} - d_{rb}|/D_{\max}}\right), \tag{7}$$

where $v_b = \|\boldsymbol{v}_b\|$ is the scalar speed of the blue and $v_{\max}$ is the maximum speed of the UCAV.

3.3 Distance advantage function

In the real scenario, the distance advantage should be related to the angle advantage of the UCAV. Additionally, given the crashing problem caused by the close distance between UCAVs, we consider the security distance in the definition of distance advantage. Therefore, we give the definition of distance advantage of r as follows:

$$F_r^d = \begin{cases} \exp^{-\frac{|d_{rb} - D_{\text{best}}|}{\max\{d_{rb} - D_{\min}, \delta\}}}, & F_r^a \geqslant F_b^a, \\ \exp^{-\frac{|D_{\max} - D_{\min}|}{\max\{d_{rb} - D_{\text{best}}, \delta\}}}, & F_r^a < F_b^a, \end{cases} \tag{8}$$

where $D_{\min}$ is the minimum relative distance with the opponent to ensure the security. δ is a very small positive number, ensuring the denominator is not equal to 0. The distance advantage function of b can be defined in the same way.

If r has the superior angle, the distance advantage increases with the decrease of the difference between d_{rb} and $D_{\min}$, since r takes the offensive situation. The denominator $\max\{d_{rb} - D_{\min}, \delta\}$ ensures d_{rb} would not be smaller than the security distance $D_{\min}$ as the distance advantage will decrease if $d_{rb} \leqslant D_{\min}$. Conversely, if r is in the defensive situation, that is, the angle advantage is smaller than that of b, the distance advantage increases with the difference between d_{rb} and D_{best}.

3.4 Altitude advantage function

During the air combat, the altitude dominance means that the UCAV holds the air supremacy. The safe flight altitude ranges from the minimum altitude $h_{\min}$ to the maximum altitude $h_{\max}$. The altitude advantage of r is

$$F_r^h = \begin{cases} 0, \ \text{if } h_r \leqslant h_{\min} \text{ or } h_r \geqslant h_{\max}, \\ 1 - \dfrac{|h_r - h_{\text{best}}|}{h_{\max} - h_{\min}}, \ \text{else}, \end{cases} \tag{9}$$

where h_r is the altitude of r. h_{best} is the optimal altitude for attacking. The definition of altitude advantage aims to ensure that the flight altitude of aircraft is as close as the optimal altitude. Similarly, we can define the altitude advantage of b in the same way.

3.5 Comprehensive advantage function

In this section, we introduce the comprehensive advantage function to evaluate the total situation advantage of aircraft. This function is characterized by the formulation of a weighted sum of the above three advantage factors. Similar to [28], the comprehensive advantage function of our paper can also adjust the weights of advantage factors under different scenarios.

$$F_r = \omega_1 F_r^d + \omega_2 F_r^a + \omega_3 F_r^h + \omega_4 F_r^v, \tag{10}$$

and it holds that $\omega_1 + \omega_2 + \omega_3 + \omega_4 = 1$.

The weight values of the air combat situational advantage should be dynamically adjusted based on the current combat scenario rather than being fixed. For example, in an offensive position (e.g., tailing the opponent), distance and speed advantages become more critical, whereas in a defensive situation (e.g., being pursued), angle and speed advantages play a greater role in evasion. Similarly, in a head-on engagement, angle advantage is paramount for post-merge positioning.

4. Individual welfare function

Since real combat requires discrete-time decision-making, and to simplify equilibrium solving, we discretize the individual welfare function. Below, we define and analyze functions from the perspective of r at time k, the related functions of b can be obtained in a similar way.

4.1 Value function

Each UCAV starts with a given value that decreases during the confrontation due to fire attacks from the opponent.

In previous studies, the hit probability of the weapon is generally set as a constant. The real hit probability every time, however, will be affected by the situation, that is, the comprehensive advantage. Accordingly, we define the value function of r at k th step as follows.

$$V_r^k = \begin{cases} 0, \ \bar{c}_{\text{capt}} = \text{true}, Nm_b^k = 1, \\ V_r^{k-1} - V_r^{k-1} d_r^k, \ \text{else}, \end{cases} \tag{11}$$

where d_r^k is the damage degree of r at step k, denoted by $d_r^k = (1 - F_r^k) Nm_b^k$. $\bar{c}_{\text{capt}}$ represents the capture condition of the opponent. Nm_b^k is the weapon number launched by b at step k. V_r^k and V_r^{k-1} are the values of r at step k and step $k-1$, respectively. F_r^k is the comprehensive advantage of r at step k, representing the probability of not being hit by the weapon. Hence, $1 - F_r^k$ is denoted as the probability of being hit, increasing with the decrease of comprehensive advantage F_r^k.

The value update is divided into two cases based on whether r falls into the capture area of b. The value of r decreases to 0 once the capture condition of b is satisfied and b uses a weapon. In the other case, r will be damaged but not destroyed at once, and the damage degree is related to the situation and the fire strategy of b.

4.2 Cost function

This paper considers only the cost of weapon consumption. Every UCAV has a limited number of weapons, and using a weapon is feasible when the fire condition is satisfied. At step k, the cost function of r is given by

$$C_r^k = Vm_r Nm_r^k, \tag{12}$$

where Vm_r is the value of weapon carried by r, Nm_r^k is the number launched at step k. At each step, only one weapon can be used at most, that is, $Nm_r^k = \{0, 1\}$.

4.3 Reward function

During the offensive process, using weapons can yield rewards for UCAVs, but the reward value depends on whether the target is within the capture area. The reward function of r is given by

$$R_r^k = \begin{cases} V_b^k Nm_r^k, \ c_{\text{capt}} = \text{true}, \\ F_r^k V_b^k Nm_r^k, \ c_{\text{capt}} = \text{false}. \end{cases} \tag{13}$$

If b is within the conical area of r, that is, $(x_b, y_b, z_b) \in \Omega_{\text{capture}}$, it is assumed that when b is destroyed, the reward for r equals the value of b. If b is attacked by r's weapon outside r's conical area, r's reward depends on the overall situation. The more dominant r is, the greater the damage to b and the higher the reward. As shown in Eq. (13), the maneuver decision affects the capture area's position, influencing fire decision-making.

4.4 Fire penalty function

To avoid the UCAV not firing when the capture condition is met and firing when the fire condition is not met, the fire penalty function of r is given by

$$Pf_r^k = \begin{cases} Vm_r, \ c_{\text{fire}} = \text{false}, Nm_r^k = 1, \\ V_b^k, \ c_{\text{capt}} = \text{true}, Nm_r^k = 0, \end{cases} \tag{14}$$

The first case represents the penalty for UAVs firing when firing conditions are not met, resulting in a waste of weapon resources. This penalty is the value of weapon in Eq. (12). The second case represents the penalty for UAVs failing to fire when capture conditions are met, resulting in missed capture opportunities. This penalty is the value of the blue. When penalty values are set too low, UCAVs show aggressive firing behavior, leading to early weapon use and resource waste. Conversely, when penalties are set too high, UCAVs become conservative, often missing firing opportunities even when conditions are favorable. The value of penalty in Eq. (14) creates a natural decision boundary where UAVs are neither overly aggressive nor conservative in their firing decisions.

4.5 Capture penalty function

To avoid the UCAV being captured by the opponent, the capture penalty function of r is given by

$$Pc_r^k = \begin{cases} \text{Pc}, \ \text{if } \bar{c}_{\text{capt}} = \text{true}, \\ 0, \ \text{else}, \end{cases} \tag{15}$$

where Pc is the capture penalty positive constant, $\bar{c}_{\text{capt}} = \text{true}$ means the UCAV enters the capture area of the opponent.

4.6 Individual welfare

Based on the above functions, the individual welfare of r at step k is defined as follows.

$$\eta_r^k = V_r^k + R_r^k + F_r^k - C_r^k - Pf_r^k - Pc_r^k. \tag{16}$$

Fire decision-making and maneuver decision-making are coupling, and their mutual influences are integrated through Eq. (16).

5. The dynamic maneuver-fire decision game model

This paper mainly focuses on the one-on-one UCAV air combat scenario, as it serves as the basis for complex air combat scenarios. We propose the dynamic maneuver-fire decision (DMFD) game model, which integrates fire and maneuver decisions within a zero-sum game framework. To simplify equilibrium solving, we define the model as a one-step game. UCAVs make decisions iteratively based on the equilibriums, enabling autonomous decision-making throughout the adversarial process.

5.1 Concepts of the DMFD game model

In this subsection, we introduce the DMFD game model with two UCAVs. The key concepts of this model are given as follows:

(1) Player: The player set of UCAVs is denoted by $N = \{r, b\}$. In the following content, we use i representing any UCAV in the player set, $i \in N$.

(2) State: At every step, we use the position coordinate, value, and remaining weapon quantity to describe the state of the UCAV. The state of i at step k is denoted by $s_i^k = [x_i^k, y_i^k, z_i^k, V_i^k, M_i^k]$, $i \in N$.

(3) Action: The action set consists of control variables, including both maneuver and fire actions. Maneuver actions involve load factors and roll angle, represented as $am_i^k = \left[n_{ix}^k, n_{iz}^k, \mu_i^k\right]$, which serve as inputs to the UCAV's motion system, controlling its maneuvers. Since our game model is one-step decision-making, we discretize maneuver actions into 15 distinct options based on these variables. Fire actions involve choosing between attack or no attack, denoted as $af_i^k = [\text{attack, keep}]$. The actions of i are listed in Table 1.

(4) Strategy: The strategy of the UCAV is composed of the maneuver action and fire action, i.e., the strategy of i at step k is $s_i^k = [am_i^k, af_i^k] = \left[n_{ix}^k, n_{iz}^k, \mu_i^k, af_i^k\right]$. So there are 30 strategies for UCAVs at every step.

(5) State transition function: There are three parts of the state transition function of i at step k.

Firstly, we can get the position transition function of i based on Eq. (1), which can be given by

$$\begin{bmatrix} x_i^k \\ y_i^k \\ z_i^k \end{bmatrix} = \begin{bmatrix} x_i^{k-1} \\ y_i^{k-1} \\ z_i^{k-1} \end{bmatrix} + \begin{bmatrix} \cos\gamma_i^k \sin\psi_i^k \\ \cos\gamma_i^k \cos\psi_i^k \\ \sin\gamma_i^k \end{bmatrix} v_i^k \Delta t, \tag{17}$$

where

$$\begin{cases} v_i^k = v_i^{k-1} + g\left(n_x^k - \sin\gamma_i^{k-1}\right) \\ \gamma_i^k = \gamma_i^{k-1} + \dfrac{g}{v_i^{k-1}}\left(n_z^k \cos\mu_i^k - \cos\gamma_i^{k-1}\right) \\ \psi_i^k = \psi_i^{k-1} + \dfrac{g n_z^k \sin\gamma_i^{k-1}}{v_i^{k-1} \cos\gamma_i^{k-1}} \end{cases}$$

Secondly, the remaining weapon quantity transition function is defined as follows:

$$M_i^k = M_i^{k-1} - Nm_i^k, \tag{18}$$

where M_i^{k-1} is the remaining weapon quantity of i at step $k-1$, Nm_i^k is the consuming weapon quantity of i at step k.

Thirdly, the value transition function is the value function in Eq. (11).

(6) Utility. In the one-on-one scenario, the two sides are fully competitive, aligning with a zero-sum game. So the utility for r is denoted by $u_r^k = \eta_r^k - \eta_b^k$ and the utility for b is denoted by $u_b^k = \eta_b^k - \eta_r^k$. The utility matrices for both sides are expressed as follows:

$$MR(k) = \begin{bmatrix} u_{1,1}^r(k) & u_{1,2}^r(k) & \cdots & u_{1,30}^r(k) \\ u_{2,1}^r(k) & u_{2,2}^r(k) & \cdots & u_{2,30}^r(k) \\ \vdots & \vdots & \ddots & \vdots \\ u_{30,1}^r(k) & u_{30,2}^r(k) & \cdots & u_{30,30}^r(k) \end{bmatrix},$$

$$MB(k) = \begin{bmatrix} u_{1,1}^b(k) & u_{1,2}^b(k) & \cdots & u_{1,30}^b(k) \\ u_{2,1}^b(k) & u_{2,2}^b(k) & \cdots & u_{2,30}^b(k) \\ \vdots & \vdots & \ddots & \vdots \\ u_{30,1}^b(k) & u_{30,2}^b(k) & \cdots & u_{30,30}^b(k) \end{bmatrix}.$$

Each entry $u_{i,j}^r(k)$ in $MR(k)$ represents the utility for r when it adopts the i th pure strategy against the j th pure strategy of b.

In finite games, a MSNE always exists and the randomness inherent in MSNE is well-suited to the uncertain and complex nature of combat environments. Therefore, we will compute the MSNE of the DMFD game model and derive the optimal maneuver-fire joint strategy to enable autonomous control of the UCAV.

5.2 Mixed strategy Nash equilibrium

Definition 1 (Mixed strategy) Given a player i with S_i as the set of pure strategies, a mixed strategy (also called randomized strategy) σ_i of player i is a probability distribution over S_i. That is, $\sigma_i : S_i \to [0, 1]$ is a mapping that assigns to each pure strategy $s_i \in S_i$ a probability $\sigma_i(s_i)$, such that

$$\sum_{s_i \in S_i} \sigma_i(s_i) = 1. \tag{19}$$

Definition 2 (Mixed strategy Nash equilibrium) Given a strategic form game $\Gamma = \langle N, (S_i), (u_i) \rangle$, a mixed strategy profile $(\sigma_1^*, \cdots, \sigma_n^*)$ is called a Nash equilibrium if $\forall i \in N$,

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geqslant u_i(\sigma_i, \sigma_{-i}^*), \ \forall \sigma_i \in \Delta(S_i). \tag{20}$$

6. PP-PSO with refinement strategy space algorithm

6.1 PP-PSO algorithm

The PSO algorithm simulates a swarm of particles updating their velocities and positions based on individual and global optimal positions to find the best solution. Inspired by nature's bird flocks and fish schools, it is widely used to solve optimization problems, with particles clustering to search for food within the solution space. The velocity and position updating equations of particle i at step $k+1$ can be expressed as [22]

$$\begin{aligned}\boldsymbol{v}_i(k+1) &= w\boldsymbol{v}_i(k)+c_1r_1[\boldsymbol{p}_i(k)-\boldsymbol{x}_i(k)]+\\ &\quad c_2r_2[\boldsymbol{g}(k)-\boldsymbol{x}_i(k)],\\ \boldsymbol{x}_i(k+1) &= \boldsymbol{x}_i(k)+\boldsymbol{v}_i(k+1)\Delta t,\end{aligned}\tag{21}$$

where c_1 and c_2 are learning factors, also known as the acceleration constants. r_1 and r_2 are random numbers between 0 and 1, reflecting the stochastic nature of the algorithm. w is the inertial factor, reflecting the effect of k th step on $(k+1)$ th step. $\boldsymbol{p}_i(k)$ is the individual historical optimal position so far. $\boldsymbol{g}(k)$ is the global optimal position in the particle swarm. Δt is the sampling time.

$$\begin{aligned}\boldsymbol{v}_{di}(k+1) &= w_d\boldsymbol{v}_{di}(k)+c_1r_1[\boldsymbol{p}_{di}(k)-\boldsymbol{x}_{di}(k)]+\\ &\quad c_2r_2[\boldsymbol{g}_d(k)-\boldsymbol{x}_{di}(k)]+c_3r_3[\boldsymbol{g}(k)-\boldsymbol{x}_{di}(k)],\end{aligned}\tag{22}$$

$$\begin{aligned}\boldsymbol{v}_{ri}(k+1) &= w_r\boldsymbol{v}_{ri}(k)+c_4r_4[\boldsymbol{p}_{ri}(k)-\boldsymbol{x}_{ri}(k)]+\\ &\quad c_5r_5[\boldsymbol{g}_r(k)-\boldsymbol{x}_{ri}(k)]+c_6r_6[\boldsymbol{g}(k)-\boldsymbol{x}_{ri}(k)]-\\ &\quad P\cdot a\cdot sign[\boldsymbol{x}_{dI}(k)-\boldsymbol{x}_{ri}(k)]\cdot\\ &\quad \exp(-b|\boldsymbol{x}_{dI}(k)-\boldsymbol{x}_{ri}(k)|).\end{aligned}\tag{23}$$

The original PSO algorithm converges quickly but is prone to getting stuck in local optima. To address this, the PP-PSO algorithm was proposed in 2006 [23], inspired by the swarm behavior of sardines and herrings. In PP-PSO algorithm, particles are divided into two categories: predators, which drive convergence, and prey, which promote diversification. The velocity and position update equations for predators and prey in the PP-PSO algorithm are defined in Eq. (22) and Eq. (23), respectively.

The position updating equations of predators and prey are

$$\boldsymbol{x}_{di}(k+1)=\boldsymbol{x}_{di}(k)+\boldsymbol{v}_{di}(k+1)\Delta t,\tag{24}$$

$$\boldsymbol{x}_{ri}(k+1)=\boldsymbol{x}_{ri}(k)+\boldsymbol{v}_{ri}(k+1)\Delta t.\tag{25}$$

d and r in the above subscripts denote predators and prey respectively. $\boldsymbol{v}_{di}(k+1)$ and $\boldsymbol{x}_{di}(k)$ are the velocity and position of i th predator at step $k+1$ and step k, respectively. $\boldsymbol{p}_{di}(k)$ is the individual historical best position for i th predator. $\boldsymbol{g}_d(k)$ is the historical best position in the predator group. $\boldsymbol{g}(k)$ is the best position in the whole particle swarm so far. The signals of prey have similar meanings.

w_d and w_r are the inertial weights of predators and prey, respectively, which are denoted by

$$w_d=0.2\cdot\exp\left(-10\frac{k}{k_{\max}}\right)+0.4,\tag{26}$$

$$w_r=w_{\max}-\frac{w_{\max}-w_{\min}}{k_{\max}}k,\tag{27}$$

where k is the iteration right now and $k_{\max}$ is the maximum number of iterations. $w_{\max}$ and $w_{\min}$ are the maximum and minimum value of w_r, respectively. The values of inertial weights are critical for the convergence ability, and regulate the balance between local and global search ability.

In Eq. (23), P indicates whether the prey escapes ($P=0$) or not ($P=1$). a and b determine how difficult it is for the prey to escape from the predator denoted by

$$a=x_{\text{span}},b=\frac{100}{x_{\text{span}}},\tag{28}$$

where x_{span} is the span of the variable. Moreover, $sign$ is a symbolic function, denoted by

$$sign(x)=\begin{cases}-1, & x<0,\\ 0, & x=0,\\ 1, & x>0.\end{cases}\tag{29}$$

I is the number of the i th prey's nearest predator, which is given by

$$I=\left\{k|\min_k|x_{dk}-x_{ri}|\right\}.\tag{30}$$

6.2 Mixed strategy Nash equilibrium and fitness function

The fitness function plays a crucial role in both the PSO and PP-PSO algorithms, evaluating the quality of a particle's position. For the DMFD game model, the fitness function of the PP-PSO algorithm must be defined based on the properties and definitions of the mixed strategy Nash equilibrium. To achieve this, we introduce the concept of the support of a mixed strategy Nash equilibrium [31].

Definition 3 (Support of a mixed strategy) Let σ_i be any mixed strategy of a player i. The support of σ_i, denoted by $\delta(\sigma_i)$, is the set of all pure strategies which have non-zero probabilities under σ_i, that is:

$$\delta(\sigma_i)=\{s_i\in S_i:\sigma_i(s_i)>0\}.\tag{31}$$

Given a mixed strategy Nash equilibrium, each player gets the same expected payoff as in the equilibrium by playing any pure strategy having positive probability in its equilibrium mixed strategy [32].

Based on the above explanation, we can define the fitness function of the PP-PSO algorithm.

At first, we define the mixed strategies for both sides as $X_i=(x_1^r,\cdots,x_{30}^r,x_1^b,\cdots,x_{30}^b)$, such that $x_l^r\geqslant 0,x_l^b\geqslant 0$, $\sum_{l=1}^{30}x_l^r=1,\sum_{l=1}^{30}x_l^b=1$. The first 30 elements form a probability distribution for the strategy of r, the last 30 elements denote a probability distribution for the strategy of b. The fitness functions of i th predator and prey are as follows:

$$f\left(X_{di}^k\right)=\max_{1\leqslant j\leqslant m}\left\{MR(j,:)\left(X_{di,31:60}^k\right)^{\mathrm{T}}-X_{di,1:30}^t MR\left(X_{di,31:60}^k\right)^{\mathrm{T}}\right\}+\max_{1\leqslant j\leqslant n}\left\{\left(X_{di,1:30}^k\right)^{\mathrm{T}}MB(:,j)-X_{di,1:30}^t MB\left(X_{di,31:60}^k\right)^{\mathrm{T}}\right\},\tag{32}$$

$$f\left(X_{ri}^k\right)=\max_{1\leqslant j\leqslant m}\left\{MR(j,:)\left(X_{ri,31:60}^k\right)^{\mathrm{T}}-X_{ri,1:30}^t MR\left(X_{ri,31:60}^k\right)^{\mathrm{T}}\right\}+\max_{1\leqslant j\leqslant n}\left\{\left(X_{ri,1:30}^k\right)^{\mathrm{T}}MB(j,:)-X_{ri,1:30}^t MB\left(X_{ri,31:60}^k\right)^{\mathrm{T}}\right\}.\tag{33}$$

In the above equations, X_{ri}^k and X_{di}^k denote mixed strategies produced by i th prey and i th predator for both sides. Additionally, these variables should satisfy the following conditions:

$$\begin{cases} X_{di,j}(k)\geqslant 0,\ \ X_{ri,j}(k)\geqslant 0, \\ \sum\limits_{j=1}^{30}X_{di,j}=1,\ \ \sum\limits_{j=31}^{60}X_{di,j}=1, \\ \sum\limits_{j=1}^{30}X_{ri,j}=1,\ \ \sum\limits_{j=31}^{60}X_{ri,j}=1. \end{cases}\tag{34}$$

According to the fitness function definition, the MSNE corresponds to the probability distribution of strategies that minimizes the fitness function, with the optimal solution occurring when the fitness value is close to zero.

6.3 Refinement of strategy space

Compared to PSO, PP-PSO avoids local optima by dividing particles into predator and prey groups, but increases computational complexity through particle classification, different velocity update equations, and group optimal position tracking. These changes reduce convergence speed. Additionally, updating the 30×30 utility matrix in the DMFD game model at each step further increases computation. To simplify MSNE calculation, we introduce domination strategies [32].

Definition 4 (Domination in Mixed Strategies) Given two mixed strategies $\sigma_i,\hat{\sigma}_i\in\Delta(S_i)$ of player i, we say σ_i strictly dominates $\hat{\sigma}_i$ if

$$u_i(\sigma_i,\sigma_{-i})>u_i(\hat{\sigma}_i,\sigma_{-i}),\forall\sigma_{-i}\in\times_{j\neq i}\Delta(S_j),\tag{35}$$

We say σ_i weakly dominates $\hat{\sigma}_i$ if

$$\begin{aligned}&u_i(\sigma_i,\sigma_{-i})\geqslant u_i(\hat{\sigma}_i,\sigma_{-i}),\ \ \forall\sigma_{-i}\in\times_{j\neq i}\Delta(S_j).\\&u_i(\sigma_i,\sigma_{-i})>u_i(\hat{\sigma}_i,\sigma_{-i}),\ \ \exists\sigma_{-i}\in\times_{j\neq i}\Delta(S_j).\end{aligned}\tag{36}$$

We say σ_i very weakly dominates $\hat{\sigma}_i$ if

$$u_i(\sigma_i,\sigma_{-i})\geqslant u_i(\hat{\sigma}_i,\sigma_{-i}),\ \ \forall\sigma_{-i}\in\times_{j\neq i}\Delta(S_j).\tag{37}$$

We say strategy $\hat{\sigma}_i$ is strongly (weakly) (very weakly) dominated by σ_i.

Definition 5 (Dominant Mixed Strategy Equilibrium) If the mixed strategy σ_i^* strongly (weakly) (very weakly) dominates all other strategies $\hat{\sigma}_i\in\Delta(S_i)$, we say σ_i^* is a strongly (weakly) (very weakly) dominant strategy of player i. A strategy profile $(\sigma_1^*,\cdots,\sigma_n^*)$ such that σ_i^* is a strictly (weakly) (very weakly) dominant strategy for player $i,\forall i\in N$, is called a strictly (weakly) (very weakly) dominant mixed strategy equilibrium.

Algorithm 1 Iterated elimination of very weakly dominated strategies

Input: current period t, utility matrix of the red team MR and the blue team MB

Output: refinement utility matrices $\overline{MR}$ and $\overline{MB}$

while there is still at least one very weakly dominated strategy **do**
$m,n\leftarrow MR.shape$;
$Mark_R\leftarrow \boldsymbol{GetMinColMat}(MR)$;
if max($\boldsymbol{SumByRow}(Mark_R)==n$ **then**
  $del_r\leftarrow$ argmax$(\boldsymbol{SumByRow}(Mark_R))$;
  $MR\leftarrow MR.DeleteRow(del_r)$;
  $MB\leftarrow MB.DeleteRow(del_r)$;
else
  Pass;
end
r $,c\leftarrow MB.shape$;
$Mark_B\leftarrow \boldsymbol{GetMinRowMat}(MB)$;
if max($\boldsymbol{SumByRow}(Mark_B)$)$==r$**then**
  $del_c\leftarrow$ argmax$(\boldsymbol{SumByRow}(Mark_B))$;
  $MR\leftarrow MR.DeleteCol(del_c)$;
  $MB\leftarrow MB.DeleteCol(del_c)$;
else
  Pass;
end
end

In game theory, players are assumed to be perfectly rational and will choose the optimal strategy. Dominated strategies, whether strictly, weakly, or very weakly dominated, are excluded. We refine the strategy space using iterated elimination of dominated strategies, ensuring the remaining profiles include all mixed strategy Nash equilibria. Eliminating weakly or very weakly dominated strategies results in smaller games. This paper applies iterated elimination of very weakly dominated strategies, as shown in Algorithm 1.

In Algorithm 1, $\boldsymbol{GetMinColMat}(MR)$ returns a Boolean matrix, where the minimum element in each column of the matrix MR equals to 1 and the other elements are 0. $\boldsymbol{GetMinRowMat}(MB)$ returns a Boolean matrix, where the minimum element in each row of matrix MB equals to 1 and the other elements are 0.

In one-on-one air combat scenarios, Fig. 6 is used to illustrate the decision-making process of UCAVs. Notably, the PP-PSO-RSS algorithm refines the strategy space of matrix games based on the concept of dominated strategies. Therefore, this algorithm is applicable to equilibrium computation in general matrix games with finite strategy spaces and is not limited to the DMFD model presented in this paper.

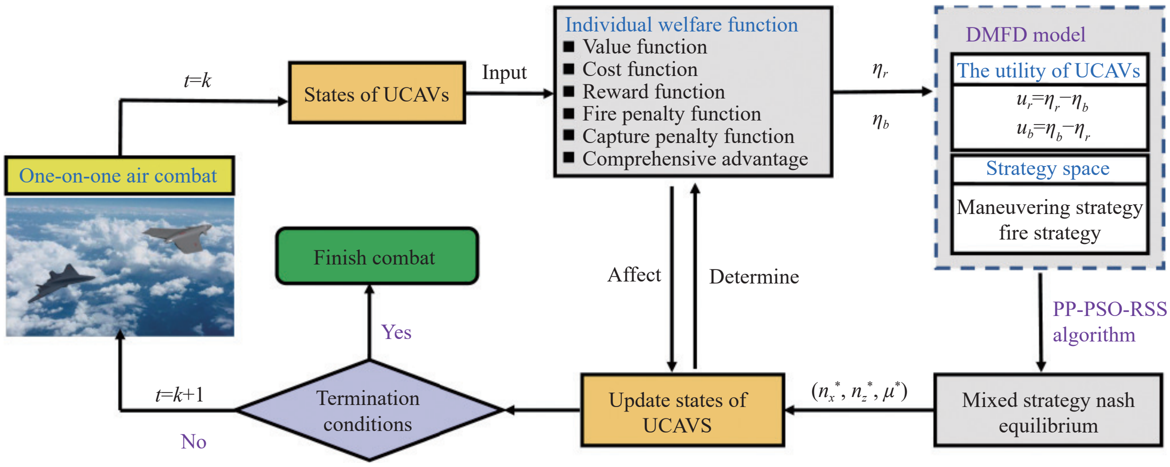


Fig. 6 Decision-making process in one-on-one air combat

7. Simulation and analysis

In this section, we operate several experiments to compare and verify the performance of our method. Specially, we mainly analyze the effectiveness of the proposed method from four aspects, including algorithm comparison, analysis of RSS effectiveness, robustness analysis, and parameter sensitivity analysis.

Air combat simulations were performed on a personal computer equipped with an AMD Ryzen 7 6800H CPU, 32 GB RAM, and an NVIDIA GeForce RTX 3060 graphics card. The terminal condition is satisfied when the value of one UCAV is less than the lower limit of value or the iteration count is larger than the setting maximum step, that is, $V \leqslant V_{\text{limit}}$ or $k \geqslant K$.

Gravity is $g = 9.8\ \text{m/s}^2$. We set the optimal attack distance $D_{\text{best}} = 800$ m, the safe distance $D_{\min} = 300$ m, and the maximum attack distance $D_{\max} = 5000$ m. The maximum speed of UCAVs is $v_{\max} = 500$ m/s. The minimum and maximum altitude are $h_{\min} = 500$ m and $h_{\max} = 15000$ m, respectively. The optimal attack altitude is $h_{\text{best}} = 8000$ m. The initial value of an UCAV is $V_0 = 4000$ and the number of weapons equipped with is $Nm = 16$. During the air combat, we set the decision time is $T = 1$ s, that is, the UCAV makes a decision every 1 s. The maximum decision step K is 90.

7.1 Algorithm comparison

7.1.1 Terminal value comparison

In this subsection, under identical initial conditions, we compare the performance of the PP-PSO-RSS algorithm against Deep Q-Network (DQN) [33], Minimax algorithm [34], and Minimax-DQN algorithms [4] for the red side, while the blue employs a statistics-based approach [35], which is a representative and effective algorithm to generate the optimal strategy based on the action frequency and membership function.

DQN is a reinforcement learning algorithm suited for discrete action spaces that optimizes policy by treating opponents as part of the environment, not guaranteeing equilibrium solutions. The Minimax algorithm solves two-player zero-sum games by determining Nash equilibrium through dual linear programming. Minimax-DQN integrates these approaches, using deep Q-networks to learn Nash equilibrium in two-player zero-sum Markov games.

Based on the UCAV value defined in Eq. (11), we use the terminal value (i.e. value of the red side at the termination moment), which reflects the outcome of the engagement, as the reward function to train DQN and Minimax DQN for 8 000 episodes. The comparative analysis of all four algorithms is then conducted from episode 8,000 to 10,000. The Performance comparison is displayed in Fig. 7.

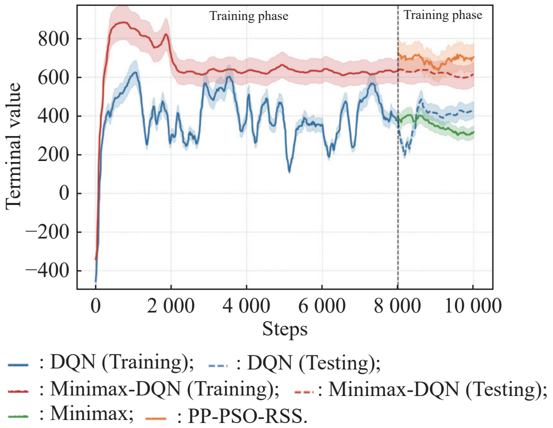


Fig. 7 Performance comparison of four algorithms

As illustrated in Fig. 7, the PP-PSO-RSS algorithm achieves the highest terminal values compared to the other three algorithms, demonstrating that our algorithm provides better maneuver selection and firing timing decisions, while more effectively avoiding being hit by opponents.

7.1.2 Win rate comparison

In this subsection, we evaluate the performance of the PP-PSO-RSS algorithm against the other five algorithms based on their win rates across 100 simulation trials.

In this simulation, the standardization value difference at the end of air combat is denoted by

$$V_r^T = (V_r^{\text{end}} - V_b^{\text{end}})/V_r^0 \tag{38}$$

where V_r^{end} and V_b^{end} refer to the terminal value of r and b, respectively, and V_r^0 is the initial value of r.

The red destroys the blue without hurt when $V_r^T = 1$, i.e., the red wins the combat. The red is dominant when $V_r^T \in (0, 1]$. Both sides reach a tie when this value equals 0.

In Fig. 8, the win rates of the red side under different algorithm combinations are represented by varying color intensities, where a deeper red indicates a higher win rate for the red side. DR in the upper right corner of each grid cell refers to the dominant rate of the red. The results demonstrate that when the blue side's algorithm remains constant, the red side achieves both highest win rates and dominance rates when employing the PP-PSO-RSS algorithm compared to other algorithms. This further validates the superiority of our proposed algorithm.

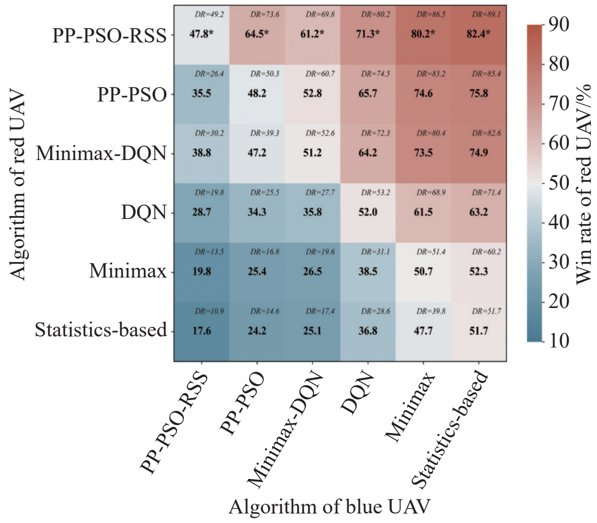
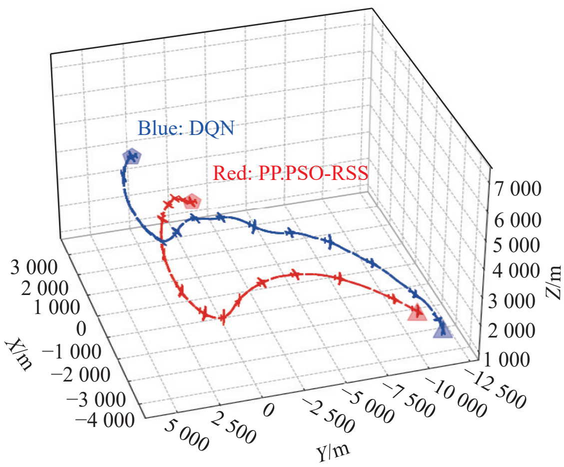


Fig. 8 Win rate comparison across different algorithms

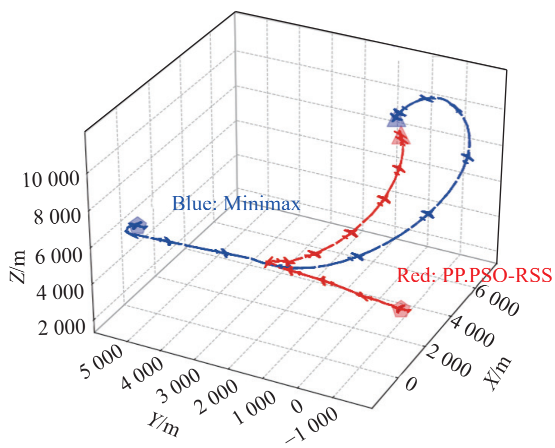
7.1.3 Trajectories comparison

In this subsection, we visualize the combat trajectories where the red employs the PP-PSO-RSS algorithm against the blue utilizing different algorithms: DQN, Minimax algorithm, Minimax-DQN, and Statistics-based method.

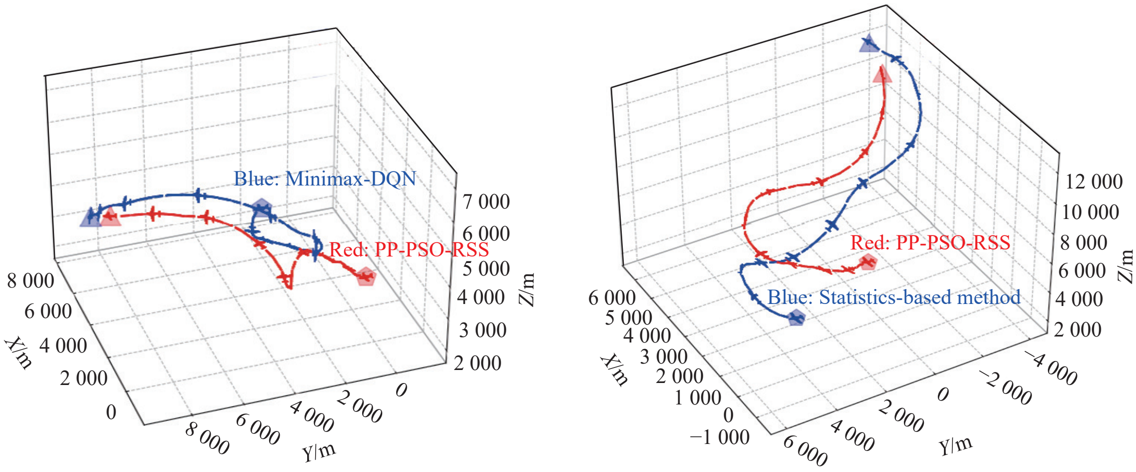
The combat trajectories of our algorithm compared to other algorithms are shown in Fig. 9. In these four cases, starting with the same initial situation, different engagement trajectories are presented, and our algorithm successfully evades the opponent's attacks while completing the capture of opponents in all scenarios. The engagement trajectories demonstrate that the maneuvering strategy possesses strong flexibility.



(a) DQN



(b) Minmax algorithm

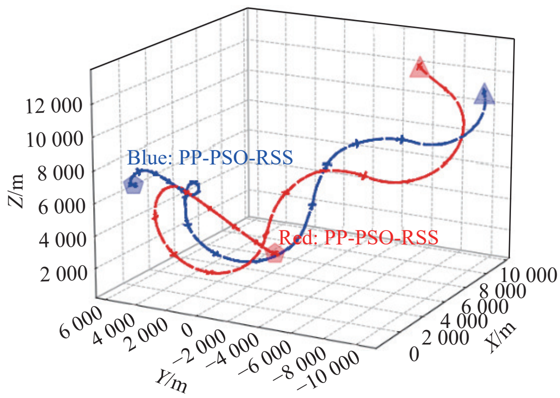


(c) Minmax-DQN (d) Statistics-based method

: Initial position of R; : Initial position of B; : End position of R; : End position of B.

Fig. 9 Air combat trajectories with the blue equipped with different algorithms

Furthermore, we conduct experiments and visualize the engagement trajectories for scenarios where both sides employ the PP-PSO-RSS algorithm. The trajectories of both sides are displayed in Fig. 10.



: Initial position of R; : Initial position of B;
: End position of R; : End position of B.

Fig. 10 Trajectories of UCAVs with PP-PSO-RSS

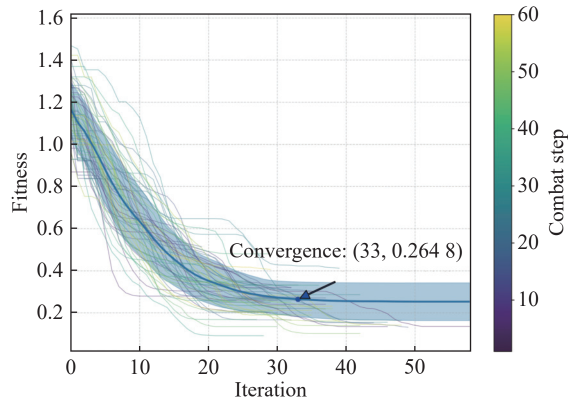
In this experiment, unlike the four scenarios in Fig. 9 where the blue is captured, neither side achieved victory within the maximum number of steps, and both sides exhibit similar trajectory patterns. These experimental results further validate the effectiveness of our proposed algorithm.

7.2 Analysis of RSS Effectiveness

In this subsection, we compare the performance of PP-PSO algorithm and PP-PSO-RSS algorithm from the perspective of the convergence and computational time to validate the effectiveness of refinement strategy space (RSS).

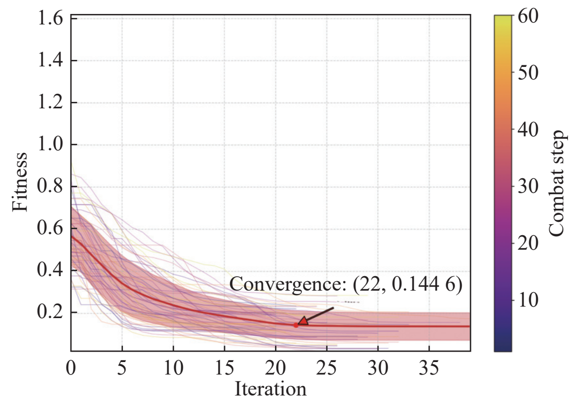
7.2.1 Convergence performance comparison

The maximum number of combat steps for UAVs is set to 60. At each decision step, equilibriums are iteratively computed using PP-PSO algorithm and PP-PSO-RSS algorithm. In Fig. 11(a) and Fig. 11(b), each curve represents the iteration process of a decision step. A smaller fitness value indicates a higher degree of equilibrium approximation.



(a) Convergence curve of PP-PSO algorithm

—: PP-PSO (Mean).



(b) Convergence curve of PP-PSO-RSS algorithm

—: PP-PSO-RSS (Mean).

Fig. 11 Convergence performance comparison

As shown in Fig. 11, the PP-PSO-RSS algorithm achieves an average convergence step of 22 with a mean fitness value of 0.1446 at convergence. In contrast, the PP-PSO algorithm requires 33 steps to converge with a mean fitness value of 0.2648. The PP-PSO-RSS algorithm demonstrates superior performance in both metrics. Therefore, compared to PP-PSO algorithm, the PP-PSO-RSS algorithm exhibits faster convergence speed and better equilibrium approximation effectiveness.

7.2.2 Computational time comparison

In Fig. 12, the blue line and red line represent the computational time of the PP-PSO algorithm and the PP-PSO-RSS algorithm, respectively. Two dashed lines represent the average computational time during the whole combat of the corresponding algorithm, respectively.

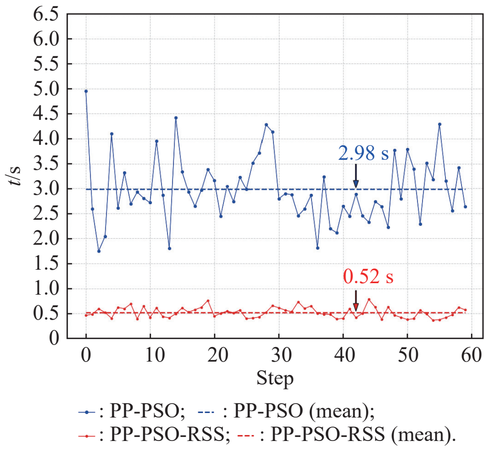


Fig. 12 Computational time comparison

As shown in Fig. 12, the average computational time of the PP-PSO-RSS algorithm is 0.52 s, while the average computational time of the PP-PSO algorithm is 2.98 s, reducing the calculation time nearly 83%.

7.3 Robustness analysis

In this subsection, the blue is set to use the statistics principle-based method as the baseline algorithm to make decisions. Under different initial conditions, we use Monte Carlo analysis to analyze the robustness of our method.

The Monte Carlo method is used to sample and perform 27000 air combat simulations. Fig. 13 shows the combat outcomes of two aircrafts under different original distances between two sides ($[1000\,\text{m}, 4000\,\text{m}]$), different original sight angles of the red ($[0, \pi]$), and different original relative velocities ($[-150\,\text{m/s}, 150\,\text{m/s}]$), that is, $v_r - v_b$. The color bar reflects the mapping from colors to the standardization value difference in Eq. (35).

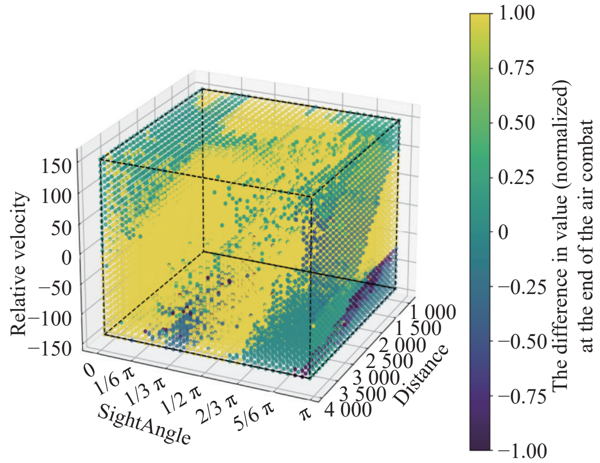


Fig. 13 The impacts of initial conditions and different methods on combat outcome

It can be seen that yellow dots are the majority, followed by green dots, and there are only a small number of purple dots in this figure. These results demonstrate the robust performance and adaptability of our proposed method across diverse initial conditions.

Fig.13 demonstrates the significant impact of initial conditions on air combat outcomes. Results show that sight angles between 0 and $\pi/3$ consistently produce favorable results, especially when combined with positive relative velocities ranging from 50 to 150 m/s. This indicates that frontal and moderately oblique engagement positions offer tactical advantages. The algorithm performs optimally at engagement distances between 500 and 2000 meters, suggesting this range constitutes the most effective combat envelope. In contrast, performance noticeably decreases at distances exceeding 3000 meters and with sight angles approaching π (rear engagement positions). These observations illustrate the effect of initial conditions on the air combat outcomes.

7.4 Parameter sensitivity analysis

In this subsection, we select seven parameters from the PP-PSO-RSS algorithm, including v_{low}, v_{up}, $Time$, $Size$, $w_{\max}$, $w_{\min}$, and $Prey_{\text{rate}}$, to analyze their sensitivity effects on the algorithm performance. We maintain all other parameters at their fixed values while varying each parameter individually through a series of values to analyze its sensitivity during the initial decision steps of air combat engagement.

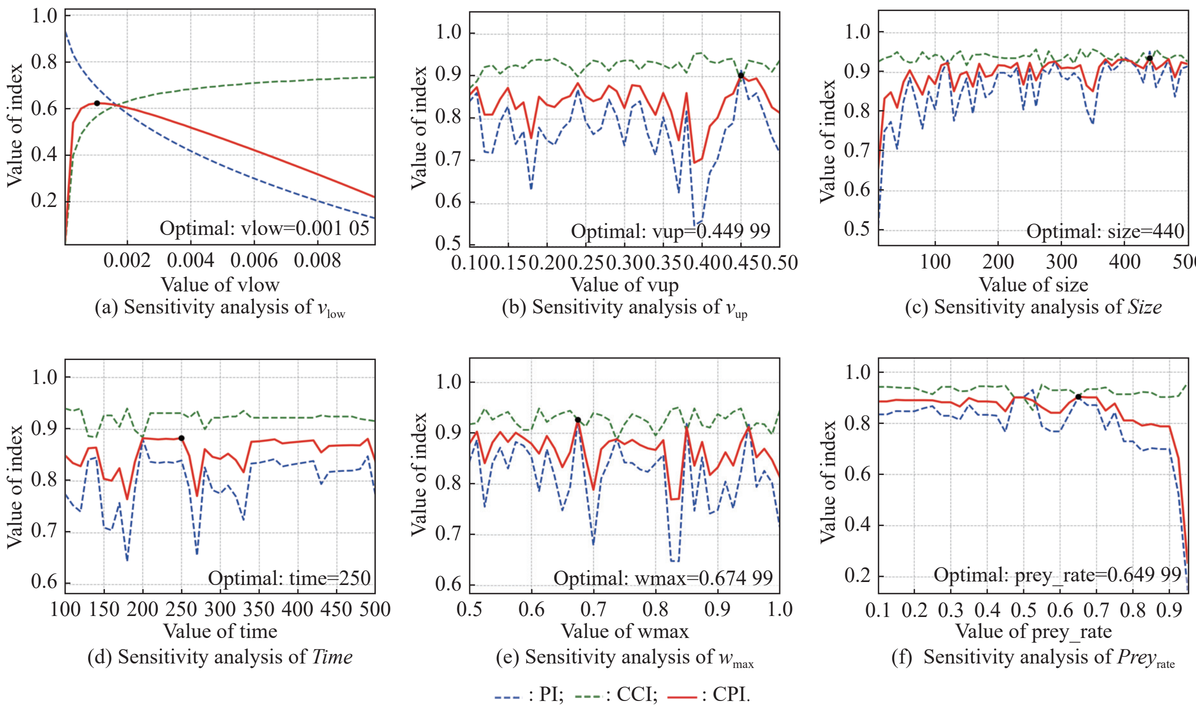
v_{low} and v_{up} are used to generate the initial speed of particles and limit the lower limit and upper limit of the particle speed. $Time$ refers to the maximum iteration times. $Size$ refers to the population size and $Prey_{\text{rate}}$ is the rate of prey particles. $w_{\max}$ and $w_{\min}$ are the maximum and minimum value of w_r in Eq. (27), respectively.

We use three indexes to evaluate the effect of parameter on the algorithm as follows:

- The precision index $PI = 1 - f/f_0$, where f is the final fitness and f_0 is the final fitness and f_0 is the initial fitness.
- The convergence capability index $CCI = 1 - K_c/K$, where K_c is the convergence step.
- The comprehensive performance index $CPI = \dfrac{2}{\dfrac{1}{PI}+\dfrac{1}{CCI}}$.

Obviously, larger PI and CCI means higher precision and faster convergence speed. However, as the iterations progress, precision typically improves, which means these two indicators are in conflict. CPI is the harmonic mean of these two indicators, which is used to balance the above two indexes. The best performance happens when the value of CPI is maximum.

During the experiments, we observe that $w_{\min}$ exhibits no sensitivity to above three performance metrics. So Fig. 14 shows the sensitivity curves of other six parameters. Based on the analysis of Fig. 14, the optimal values of these parameters are indicated by black dots in the figures, which are marked in the bottom right corner of each subplot.



(a) Sensitivity analysis of v_{low} (b) Sensitivity analysis of v_{up} (c) Sensitivity analysis of *Size*

(d) Sensitivity analysis of *Time* (e) Sensitivity analysis of $w_{\max}$ (f) Sensitivity analysis of $Prey_{\text{rate}}$

Fig. 14 Parameters sensitivity curves

8. Conclusions

This paper introduced the DMFD game model, which comprehensively addresses maneuver and fire decision-making for UCAVs in dynamic confrontations, analyzing the interaction between these strategies. The model provides a complete decision-making framework for air combat. To compute the mixed strategy Nash equilibrium, we proposed the PP-PSO-RSS algorithm, which can also be applied to non-zero-sum and multi-player games. Several experiments demonstrated the effectiveness of our method in terms of combat performance, robustness, and computational time.

In future work, we will conduct research on the impact of UCAV performance variables on win-rates, and expand our study to cluster combat scenarios, covering pre-mission deployment, maneuver decision-making, and offensive-defensive decision-making through aerial fire support to broaden the application prospects of our research.

Additionally, as our current model is built upon complete information scenarios without communication constraints and primarily focuses on maneuver-fire decision-making. Future research could extend to multi-vs-multi scenarios to introduce communication constraints, apply Bayesian game theory or opponent modeling to address incomplete information challenges, and implement advanced filtering techniques to handle sensor noise issues in air combat.

References

[1] HUANG C Q, DONG K S, HUANG H Q, TANG S Q, ZHANG Z R. Autonomous air combat maneuver decision using Bayesian inference and moving horizon optimization. Journal of Systems Engineering and Electronics, 2018, 29(1): 86–97.

[2] AUSTIN F, CARBONE G, FALCO M, HINZ H, LEWIS M. Game theory for automated maneuvering during air-to-air combat. Journal of Guidance, Control, and Dynamics, 1990, 13(6): 1143–1149.

[3] DUAN Z K, XU G J, LIU X, MA J Y, WANG L Y. Optimal confrontation position selecting games model and its application to one-on-one air combat. Defence Technology, 2024, 31: 417–428.

[4] JI M D, XU G J, DUAN Z K, et al. Cooperative pursuit with multiple pursuers based on deep minimax Q-learning. Aerospace Science and Technology, 2024, 146: 108919.

[5] LI S Y, CHEN M, WANG Y H, WU Q X. Air combat decision-making of multiple UCAVs based on constraint strategy games. Defence Technology, 2022, 18(3): 368–383.

[6] MENG Y F, XU J C, HE J H, et al. A cluster UAV inspired honeycomb defense system to confront military IoT: A dynamic game approach. Soft Computing, 2023, 27(2): 1033–1043.

[7] BARDHAN R, GHOSE D. Differential games guidance for heading angle consensus among unmanned aerial vehicles. Journal of Guidance, Control, and Dynamics, 2019, 42(11): 2568–2575.

[8] MCGREW J S. Real-time maneuvering decisions for autonomous air combat. Cambridge, UK: Massachusetts Institute of Technology, 2008.

[9] CRUMPACKER J B, ROBBINS M J, JENKINS P R. An approximate dynamic programming approach for solving an air combat maneuvering problem. Expert Systems with Applications, 2022, 203: 117448.

[10] LI Z W, et al. UAV networks against multiple maneuvering smart jamming with knowledge-based reinforcement learning. IEEE Internet of Things Journal, 2021, 8(15): 12289–12310.

[11] WILLIAMS K R, SCHLOSSMAN R, WHITTEN D, et al. Trajectory planning with deep reinforcement learning in high-level action spaces. IEEE Transactions on Aerospace and Electronic Systems, 2023, 59(3): 2513–2529.

[12] ZHAN G, ZHANG K, LI K, PIAO H Y. UAV maneuvering decision-making algorithm based on deep reinforcement learning under the guidance of expert experience. Journal of Systems Engineering and Electronics, 2024, 35(3): 644–665.

[13] VIRTANEN K, KARELAHTI J, RAIVIO T. Modeling air combat by a moving horizon influence diagram game. Journal of Guidance, Control, and Dynamics, 2006, 29(5): 1080–1091.

[14] ZHONG L, TONG M A, ZHONG W, ZHANG S Y. Sequential maneuvering decisions based on multi-stage influence diagram in air combat. Journal of Systems Engineering and Electronics, 2007, 18(3): 551–555.

[15] LI Z W, CHANG Y Z, KOU Y X, YANG H Y, XU A, LI Y. Approach to WTA in air combat using IAFSA-IHS algorithm. Journal of Systems Engineering and Electronics, 2018, 29(3): 519–529.

[16] XU J W, DENG Z H, SONG Q, et al. Multi-UAV counter-game model based on uncertain information. Applied Mathematics and Computation, 2020, 366: 124684.

[17] ZHA W Z, CHEN J, PENG Z H. Dynamic multi-team antagonistic games model with incomplete information and its application to multi-UAV. IEEE-CAA Journal of Automatica Sinica, 2015, 2(1): 74–84.

[18] CHEN C, MO L, LV M L, et al. Enhanced missile hit probability Actor-Critic algorithm for autonomous decision-making in air-to-air confrontation. Aerospace Science and Technology, 2024, 109285.

[19] SUN Z X, PIAO H Y, YANG Z, et al. Multi-agent hierarchical policy gradient for air combat tactics emergence via self-play. Engineering Applications of Artificial Intelligence, 2021, 98: 104112.

[20] PORTER R, NUDELMAN E, SHOHAM Y. Simple search methods for finding a Nash equilibrium. Games and Economic Behavior, 2008, 63(2): 642–662.

[21] CHEN X, DENG X T, TENG S H. Settling the complexity of computing two-player Nash equilibria. Journal of the ACM (JACM), 2009, 56(3): 1–57.

[22] KENNEDY J, EBERHART R. Particle swarm optimization. Proc. of the IEEE International Conference on Neural Networks, 1995: 1942-1948.

[23] HIGASHITANI M, ISHIGAME A, YASUDA K. Particle swarm optimization considering the concept of predator-prey behavior. Proc. of the IEEE International Conference on Evolutionary Computation, 2006: 434-437.

[24] REN Z, ZHANG D, TANG S, et al. Cooperative maneuver decision making for multi-UAV air combat based on incomplete information dynamic game. Defence Technology, 2023, 27: 308–317.

[25] FU X W, PAN J, WANG H X, et al. A formation maintenance and reconstruction method of UAV swarm based on distributed control. Aerospace Science and Technology, 2020, 104: 105981.

[26] YANG Z, YUAN Z C, WANG X Y, et al. Autonomous control of UAV trajectory based on RHC-Radau method in complex penetration combat environment. Aerospace Science and Technology, 2024, 146: 108915.

[27] JIANG F L, XU M Q, LI Y Q, et al. Short-range air combat maneuver decision of UAV swarm based on multi-agent Transformer introducing virtual objects. Engineering Applications of Artificial Intelligence, 2023, 123: 106358.

[28] ZHANG J D, YANG Q M, SHI G Q, et al. UAV cooperative air combat maneuver decision based on multi-agent reinforcement learning. Journal of Systems Engineering and Electronics, 2021, 32(6): 1421–1438.

[29] ZHANG J D, YU Y F, ZHENG L H, et al. Situational continuity-based air combat autonomous maneuvering decision-making. Defence Technology, 2023, 29: 66–79.

[30] CRUMPACKER J B, ROBBINS M J, JENKINS P R. An approximate dynamic programming approach for solving an air combat maneuvering problem. Expert Systems with Applications, 2022, 203: 117448.

[31] LI Y F, LYU Y X, SHI J P, et al. Autonomous maneuver decision of air combat based on simulated operation command and FRV-DDPG algorithm. Aerospace, 2022, 9(11): 658.

[32] NARAHARI Y. Game Theory and Mechanism Design. World Scientific, 2014.

[33] CAO Y, KOU Y X, Li Z W, et al. Autonomous maneuver decision of UCAV air combat based on double deep Q network algorithm and stochastic game theory. International Journal of Aerospace Engineering, 2023, 2023(1): 3657814.

[34] LI Z S, XU G J, WANG L Y. A generalizable autonomous maneuvering decision-making method for UCAV air combat combining PER-D3QN and zero-sum Markov game. Proc. of the International Conference on Autonomous Unmanned Systems, 2023: 97-107.

[35] GUO H F, HOU M Y, ZHANG Q J, et al. UCAV robust maneuver decision based on statistics principle. Acta Armamentarii, 2017, 38(1): 160–167.

Biographies

DUAN Zekun was born in 1997. She is currently pursuing the Ph.D. degree at Northwestern Polytechnical University. Her research interests include game theory and its application in unmanned systems.
E-mail: zekun.duan@mail.nwpu.edu.cn

XU Genjiu was born in 1978. He received the Ph.D. degree from University of Twente. He is a professor at Northwestern Polytechnical University. His research interests include cooperative game theory, mechanism design, and unmanned systems.
E-mail: xugenjiu@nwpu.edu.cn

LI Zesheng was born in 2000. He received his B.S. degree in information and computing science from Huazhong Agricultural University. He is currently studying for a master's degree at Northwestern Polytechnical University. His main research interests include swarm intelligent decision making and multi-agent systems.
E-mail: zeshengli@mail.nwpu.edu.cn

MENG Binbin was born in 1987. He received the Ph.D. degree from National University of Defense Technology. He is a full assistant research fellow in Chinese Academy of Military Science. His main research interests include game theory and its application in AI.
E-mail: mengbinbin@nudt.edu.cn